



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO



Sesión de Laboratorio 6:

ECDSA Y ECDH (MUNDO REAL)



SELECTED TOPICS OF CRYPTOGRAPHY

Profesora || **Sandra Díaz Santiago**
Grupo || **7CM1**
Alumno || **Areli Alejandra Guevara Badillo**

24 de marzo de 2026

EJERCICIOS DE PROGRAMACIÓN

1. ECDSA con curvas elípticas estándar

Elaborado por: Héctor Rigel Ocaña Castro

Para la implementación se debe desarrollar un programa utilizando una biblioteca criptográfica y curvas elípticas estándar sobre campos primos del NIST, capaz de ejecutar exclusivamente una de tres funciones: generación de par de claves (almacenando clave privada y pública en archivos de texto Base64 separados), generación de firma (aplicando hash SHA2 y guardando el par (r, s) en un archivo) y verificación de firma (leyendo clave pública, mensaje y firma para retornar un booleano de validez), permitiendo al usuario definir los nombres de los archivos en cada procedimiento.

Ejemplo de par de llaves:

```
pub.txt    X
1 LS0tLS1CRUdJTiBQVklwJSUMgS0ZlS0tLS0KUzRvd0VWUhIb1pJemowQ0FRWUl1b1pJemowREFRY0RRZ0FFRIWUXEYm53L2YxOGZianVrQ0Irb2xDcWNkKwpBc2t2eD0JekFvZjA0Y255T3RKMydxY2VYTzRwLHJfNFoiZFpmaEHM2o2M3c3R09jQjZ4am1LSXF8PT0KLS0tLS1FTkoqUFVCtLEIEtFWStLS0tCg==

priv.txt   X
1 LS0tLS1CRUdJTiBQVklwQVRFIETfWS0tLS0tCk1JR0h8b0Z0VTUjNR05cudTTTQ5QxdFR0NdCUdT
TTQ5QXdfSE3HHdhdd0lCVFRZ2kb0Zldlczk5ZGRncm82cGgKMUNSZU9UT2SWagi2dmTxZTNjZTZK
ZW00WLjPAP3BTkNBQVFUULjDcj11kZQ5L1h4OXVnPlFJaVnPVutweHY0QWP5U3p1WTNNQ2gvVGh5
Zkk2MG5idFpANmM3aWxsZHpbomwbxcTFZEXlUHJMhRNZNkdIckdPWm9pbwotLS0tLUVORCBQClW
QVRFIETfWS0tLS0tCg==
```

PROBLEMAS **TERMINAL** CONSOLA DE DEPURACIÓN PUERTOS EXPLORADOR SALIDA

Python + - [] [X] ... [] [X]

```
● PS C:\Users\Arelí Guevara\OneDrive - Instituto Politecnico Nacional\Documents\Escuela\Selected Topics of Cryptography\Practicas\Practica 6\src> &"C:\Users\Arelí Guevara\AppData\Local\Programs\Python\Python314\python.exe" "c:\Users\Arelí Guevara\OneDrive - Instituto Politecnico Nacional\Documents\Escuela\Selected Topics of Cryptogra
phy\Practicas\Practica 6/src/ECDsa.py"
1: P-224, 2: P-256, 3: P-384, 4: P-521
Elige curva (1-4): 2
Ingrese el nombre del archivo para guardar la clave privada: priv.txt
Ingrese el nombre del archivo para guardar la clave publica: pub.txt
Ingrese el nombre del archivo con el mensaje a firmar: odaGato.txt
Ingrese el nombre del archivo a guardar la firma: odaGato.enc
Verificación de la firma: VÁLIDA
```

2. ECDH con curvas elípticas estándar

Elaborado por: Areli Alejandra Guevara Badillo

Para el esquema ECDH, se debe simular el acuerdo de claves entre dos entidades permitiendo la selección de curvas NIST (P-224, P-256, P-384 o P-521), donde las claves públicas parciales (aG y bG) se guardan o imprimen en Base64 para su intercambio, calculando posteriormente el secreto compartido $K = abG$ y utilizando una función de derivación de claves (KDF) basada en SHA2 para obtener una clave secreta k de 256 bits, mostrando tanto el secreto acordado como la clave derivada en formato Base64.

Ejecución:

```
PS C:\Users\Arelí Guevara\OneDrive - Instituto Politecnico Nacional\Documents\Escuela\Selected Topics of Cryptography\Practicas\Practica 6\src> &
ra\AppData\Local\Programs\Python\Python314\python.exe" "c:\Users\Arelí Guevara\OneDrive - Instituto Politecnico Nacional\Documents\Escuela\Selecte
phy\Practicas\Practica 6\src\ECDH.py"
1: P-224, 2: P-256, 3: P-384, 4: P-521
Elige curva (1-4): 3

Alice (aG): BHSsJ8GrhKqM7jqoMnhxE0SNaAcnWA0fa0/yVzehLQjbrFuxKFq8GnTAbtDsPN2YDCSY+ymclwN+rV/+zwZmmZd/qTyDu04StMJP98eceBkhtOnWUe3tt/rnDjCog==
Bob (bG): BNH6TL0ZuT530NEMCVkLTA3NwfkPgbgCqLDrKLOHQDTUHNjdwFUjgaaab6AVkt4q3k8dajERSQZxng0mrrG87VA0otwYGV2mgg80JEYXdxj4VUMkaokQ1Xk50BtQ7Cwg==

Secreto Compartido K (Base64): YhdZ9LPsMpF0XfRLVY/BuIWS9LgGfPqiXg1YybTKJCoS30dLty1Kv/6fHSJ1EXC
Clave Derivada k (256 bits): JRDLmkhpcBF3GUS5fxLhG6CYHuU+1fDTHkRwmcDt054=
```

PREGUNTAS

1. ¿Cuáles son los parámetros dados en el estándar para cada una de las curvas elípticas: P-224, P-256, P-384, P-521?

Curva	p	n	a	b	Gx	Gy
P-224	$2^{224} - 2^{96} + 1$	0xffffffff ffffffff ffffffff ffffffff 00000000 00000000 00000001	-3	0xb4050a85 0c04b3ab f5413256 5044b0b7 d7bfd8ba 270b3943 2355ffb4	0xb70e0cbd 6bb4bf7f 321390b9 4a03c1d3 56c21122 343280d6 115c1d21	0xbd376388 b5f723fb 4c22dfe6 cd4375a0 5a074764 44d58199 85007e34
P-256	$2^{256} - 2^{224} + 2^{192} + 2^{96} - 1$	0xffffffff 00000000 FFFFFFFF ffffffff bce6faad a7179e84 f3b9cac2 fc632551	-3	0x5ac635d8 aa3a93e7 b3ebbd55 769886bc 651d06b0 cc53b0f6 3bce3c3e 27d2604b	0x6b17d1f2 e12c4247 f8bce6e5 63a440f2 77037d81 2deb33a0 f4a13945 d898c296	0x4fe342e2 fe1a7f9b 8ee7eb4a 7c0f9e16 2bce3357 6b315ece cbb64068 37bf51f5
P-384	$2^{384} - 2^{128} - 2^{96} + 2^{32} - 1$	0xffffffff ffffffff ffffffff ffffffff ffffffff ffffffff c7634d81 f4372ddf 581a0db2 48b0a77a ecec196a ccc52973	-3	0xb3312fa7 e23ee7e4 988e056b e3f82d19 181d9c6e fe814112 0314088f 5013875a c656398d 8a2ed19d 2a85c8ed d3ec2aef	0xaa87ca22 be8b0537 8eb1c71e f320ad74 6e1d3b62 8ba79b98 59f741e0 82542a38 5502f25d bf55296c 3a545e38 72760ab7	0x3617de4a 96262c6f 5d9e98bf 9292dc29 f8f41dbd 289a147c e9da3113 b5f0b8c0 0a60b1ce 1d7e819d 7a431d7c 90ea0e5f
P-521	$2^{521} - 1$	0x1ff ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff ffffffff fffffffa 51868783 bf2f966b 7fcc0148 f709a5d0 3bb5c9b8 899c47ae bb6fb71e 91386409	-3	0x051 953eb961 8e1c9a1f 929a21a0 b68540ee a2da725b 99b315f3 b8b48991 8ef109e1 56193951 ec7e937b 1652c0bd 3bb1bf07 3573df88 3d2c34f1 ef451fd4 6b503f00	0xc6 858e06b7 0404e9cd 9e3ecb66 2395b442 9c648139 053fb521 f828af60 6b4d3dba a14b5e77 efe75928 fe1dc127 a2ffa8de 3348b3c1 856a429b f97e7e31 c2e5bd66	0x118 39296a78 9a3bc004 5c8a5fb4 2c7d1bd9 98f54449 579b4468 17afbd17 273e662c 97ee7299 5ef42640 c550b901 3fad0761 353c7086 a272c240 88be9476 9fd16650

2. ¿Cuál es el nivel de seguridad para cada una de las curvas elípticas de la pregunta 1? ¿Qué significa que tengan dicho nivel de seguridad?

El nivel de seguridad indica la cantidad de operaciones computacionales ($\sim 2^n$) requeridas para romper la criptografía mediante el mejor ataque conocido.

Por ejemplo, 128 bits significa que se necesitarían aproximadamente 2^{128} operaciones para comprometer la clave.

Curva	Nivel de Seguridad (bits)
P-224	112 bits
P-256	128 bits
P-384	192 bits
P-521	256 bits

3. Especifique la biblioteca criptográfica y el lenguaje de programación que utilizó para implementar sus programas.

- Lenguaje: Python 3.14
- Biblioteca: cryptography

4. ¿Cómo estableció los parámetros de la curva en su programa? ¿Qué función o conjunto de instrucciones utilizó para este propósito?

Los parámetros de la curva se establecieron mediante clases predefinidas de la biblioteca cryptography.

Constructor de clases `ec.SECPXXR1()` que cargan automáticamente todos los parámetros NIST (p , a , b , G , n) desde el estándar.

```
curvas = {
    '1': ec.SECP224R1(),
    '2': ec.SECP256R1(),
    '3': ec.SECP384R1(),
    '4': ec.SECP521R1()
}
curva = curvas.get(opcion, ec.SECP256R1())
```

5. ¿Cuáles son las instrucciones explícitas para generar el par de claves ECDSA? Explique cada paso. Si está usando una función, explique cuáles son los parámetros requeridos.

Función utilizada: `Generacion(Curva, archivo, archivo2)`

Parámetro	Descripción
Curva	Objeto de curva NIST (<code>ec.SECP256R1()</code> , etc.)
archivo	Nombre de archivo para clave privada (Base64)
archivo2	Nombre de archivo para clave pública (Base64)

1. `ec.generate_private_key(Curva)`: Genera clave privada aleatoria en $[1, n-1]$
2. `private_bytes(...)`: Serializa clave privada a formato PEM/PKCS#8
3. `priv_key.public_key()`: Calcula clave pública $Q = d \cdot G$
4. `public_bytes(...)`: Serializa y codifica clave pública

6. ¿Cuáles son las instrucciones explícitas para realizar la generación de firma ECDSA? Explique cada paso. Si está usando una función, explique cuáles son los parámetros requeridos.

Función utilizada: `firma(Archivo_Kpriv, Archivo_firma, m)`

Parámetro	Descripción
Archivo_Kpriv	Archivo con clave privada en Base64
Archivo_firma	Archivo de salida para firma (r, s)
m	Archivo con mensaje original a firmar

1. Lectura del mensaje m
2. Cálculo del hash: $e = \text{SHA256}(m)$ usando `hashes.Hash`
3. Carga de clave privada: decodificar Base64 objeto `private_key`
4. Firma: `priv_key.sign(digest, ec.ECDSA(Prehashed(SHA256())))`
 1. Genera nonce aleatorio k
 2. Calcula $R = k \cdot G$, obtiene $r = R.x \bmod n$
 3. Calcula $s = k^{-1} \cdot (e + d \cdot r) \bmod n$
5. `decode_dss_signature()`: Extrae componentes (r, s) de firma DER
6. Guarda r y s

7. ¿Cuáles son las instrucciones explícitas para realizar la verificación de firma ECDSA? Explique cada paso. Si está usando una función, explique cuáles son los parámetros requeridos.

Función utilizada: `Verificacion(Archivo_Kpub, Archivo_firma, m)`

Parámetro	Descripción
Archivo_Kpub	Archivo con clave pública en Base64
Archivo_firma	Archivo con firma (r, s) en Base64
m	Archivo con mensaje original a verificar

1. Lectura del mensaje m y cálculo de $e = \text{SHA256}(m)$
2. Carga de clave pública: decodificar Base64 objeto `public_key`
3. Lectura de firma: enteros r y s
4. `encode_dss_signature(r, s)`: Reconstruye firma en formato DER
5. Verificación: `pub_key.verify(firma, digest, ec.ECDSA(Prehashed(SHA256())))`
 - Calcula $w = s^{-1} \bmod n$
 - Calcula $u_1 = e \cdot w \bmod n$, $u_2 = r \cdot w \bmod n$
 - Calcula $R' = u_1 \cdot G + u_2 \cdot Q$
 - Valida si $r \equiv R'.x \bmod n$
6. Retorna True si verifica, False si lanza `InvalidSignature`

8. ¿Cuál es la función o conjunto de instrucciones que utilizó en su programa para ejecutar el protocolo ECDH? ¿Cuál es el formato original de la salida (binario, entero, hexadecimal)?

- **Función:** `private_key.exchange(ec.ECDH(), public_key)`
- **Cálculo interno:** $K = abG$ (multiplicación escalar del punto)
- **Formato original de salida:** La biblioteca devuelve bytes puros del secreto compartido.